

```

print file type,size
#include<stdio.h>
#include<string.h>
#include<sys/stat.h>
void main()
{
    Struct stat filestat;
    FILE * f1;
    char s1[100];
    Printf("enter file name");
    Scanf("%s",&s1);
    f1=fopen(s1,"r");

printf("size=%ld",filestat.st_size);
printf("type=");
if(strstr(s1,".html"))
printf("html type");

if(strstr(s1,".csv"))
printf("csv type");

if(strstr(s1,".php"))
printf("php type");

```

- Write a 'C' program to illustrating seek() system call.
- Write a 'C' program to Print all environment variables.

```

#include<stdio.h>
int main(int argc, char *argv,char*
envp)
{
    char ** env;
    for(env=envp;*env!=0 ;env++)
    {
        char * thisEnv=*env;

printf("%s\n",thisEnv);
    }
}

```

- Write a 'C' program to illustrate use of time system call.
- Write a 'C' program to find weather a given File is present in Current Directory

```

#include<sys/stat.h>
#include<sys/types.h>
#include<dirent.h>
#include<string.h>
int main()
{
    DIR *dr;
    struct dirent *en;
    String s1[20];

```

```

        dr=opendir(".");
        printf("enter file name to
match");
        scanf("%s", &s1);
        if (dr)
        {
while((en=readdir(dr))!=NULL)
        {
if(strstr(en->d_name,s1)!=NULL)
        printf("\n %s",
en->d_name);
        }
        }
    }

```

- Write a 'program that copies its standard input to standard output.

```

#include<stdio.h>
int main()
{
    int c;

    while((c=getchar())!=EOF)
    {
        putchar(c)
    }
}

```

- Explain the behavior of following C'

program.

```
char str1[]="Linux";
Main ()
{
    int
fd[2];

    char str 2[10];
    pipe(fd);
    if(fork()== 0)
        write(fd[1],str1,5)
    else
        read(fd[0],str2,5);
```

->

21 explain the behaviour of following C program

```
#include<signal.h>
main(intargc char *argv[])
{
    charbuf[256];if(argc!=1)
        signal(.SIGCLD,SIG-IGN);

while(read(0,buf,256));if(fork()==0)
    exit(0);
```

-> the program ignores the SIGCHLD signal which is sent when a child process terminates

this allows the parent process to

continue running without waiting for child process to finish

- Explain the behavior of following C' program.

```
->          main ()
                        {
                                int
fd, char buf [1024];

fd=create("junk",0666);
                                lseek(fd,2001,2);
                                write(fd,"hello",5);
                                close fd();
                        }
```

->create is likely creat()  
0666 is the file permission mode  
the file "junk" is created with read/write permissions  
behaviour:  
a new file named "junk" is created with specified permission  
the lseek() moves the end of the file  
the file "junk" have a total size  
=2001+5=2006 bytes